PROJECT REPORT

PLANT LEAF DISEASE DETECTION USING DEEP NEURAL NETWORKS

A Software-Based Deep Learning Image Classification Application

Prepared by: _____

Department / Institution: _____

Academic Year: _____

Note: Add actual training metrics and screenshots after successfully running the project.

Table of Contents

Abstract

1. Introduction

2. Problem Statement

3. Objectives

4. Existing System

5. Proposed System

6. System Requirements

7. Methodology

8. System Architecture

9. Model Design

10. Implementation

11. Testing Plan

12. Results and Discussion

13. Limitations

14. Future Enhancements

15. Conclusion

References

Abstract

This project proposes a software-based plant leaf disease classification system using a Deep Neural Network (DNN). A user uploads a plant-leaf image through a simple web interface, and the trained model predicts one of the disease or health categories represented in the training dataset. The system uses Python, TensorFlow/Keras, image preprocessing, and Streamlit. The project is intended as an educational prototype that demonstrates image classification and deployment of a machine-learning model. Final accuracy and other evaluation values must be added only after the model is trained and tested.

1. Introduction

Plant diseases can affect crop quality and yield. Visual inspection is useful, but identifying symptoms can be difficult for beginners and may require experience. Image classification with deep learning can learn visual patterns from labelled leaf images and provide a preliminary category prediction.

This project builds a small web application that accepts an image, resizes and normalizes it, passes it to a trained neural network, and displays the predicted class and model confidence. The prediction is not a substitute for expert agricultural advice.

2. Problem Statement

Identifying plant leaf conditions from images manually can be time-consuming and inconsistent. A simple, accessible software tool is needed to demonstrate how a deep-learning model can classify leaf images and present its output through a user-friendly interface.

3. Objectives

• Build an image-classification model using TensorFlow/Keras.

• Prepare and preprocess labelled plant-leaf images.

• Train and validate a convolutional neural network.

• Develop a Streamlit interface for image upload and prediction.

• Evaluate the model using validation metrics and document the implementation.

• Publish source code and documentation in a GitHub repository.

4. Existing System

Traditional identification depends on visual inspection and the experience of the person examining the plant. It can be difficult to apply consistently across many images. Some digital tools require a dedicated mobile application or a specialised service.

5. Proposed System

The proposed system uses a trained convolutional neural network (CNN), a type of deep neural network designed for image data. The web interface accepts JPG, JPEG, or PNG files. The image is converted to RGB, resized to 128 × 128 pixels, and normalized to values between 0 and 1. The model returns class probabilities; the application displays the class with the highest predicted probability.

Scope note: the implementation described in the accompanying starter code selects tomato-related classes from the dataset. The exact dataset repository, available label names, and image-column format must be verified when running the training script.

6. System Requirements

Hardware: Windows laptop or desktop; an internet connection is needed to download the dataset and Python packages. A GPU is optional; CPU training may take longer.

Software: Python 3.10 or 3.11 recommended, Visual Studio Code, TensorFlow/Keras, NumPy, Pillow, scikit-learn, Hugging Face Datasets (for dataset loading), and Streamlit.

7. Methodology

Step 1 — Dataset collection: obtain a labelled plant-leaf image dataset such as PlantVillage and verify its usage terms and class labels.

Step 2 — Preprocessing: convert images to RGB, resize to 128 × 128 pixels, and scale pixel values to the range 0–1.

Step 3 — Data split: separate the examples into training and validation subsets. Use stratified splitting where supported so each class is represented in both subsets.

Step 4 — Model construction: create a CNN with convolution layers, max-pooling, global average pooling, a dense layer, dropout, and a softmax output layer.

Step 5 — Training: compile with the Adam optimizer and sparse categorical cross-entropy loss, then train for a small number of epochs as a baseline.

Step 6 — Evaluation: record validation loss and accuracy. For a stronger evaluation, also calculate a confusion matrix and per-class precision, recall, and F1-score using a separate test set.

Step 7 — Deployment: save the trained model and class names, then load them in a Streamlit application for prediction.

8. System Architecture

Input leaf image → RGB conversion and resize → pixel normalization → trained CNN/DNN → softmax class scores → predicted label and confidence → Streamlit result screen.

9. Model Design

The baseline model uses three convolutional stages to learn visual features, followed by global average pooling and dense layers for classification. Dropout is included to help reduce overfitting. The softmax output produces a probability distribution across the selected classes.

Example layer sequence: Input (128 × 128 × 3) → Conv2D(16) → MaxPooling2D → Conv2D(32) → MaxPooling2D → Conv2D(64) → GlobalAveragePooling2D → Dense(64) → Dropout(0.3) → Softmax output.

10. Implementation

train_model.py: loads and filters the dataset, preprocesses images, creates the CNN, trains and evaluates it, and saves the model and class names.

app.py: provides the Streamlit user interface, accepts an uploaded image, applies the same preprocessing used during training, and displays the predicted class and confidence.

requirements.txt: lists the Python packages needed to recreate the environment.

Important: the training and app scripts must use exactly the same image size, normalization, and class-name order. Run the training script successfully before starting the app.

11. Testing Plan

• Verify that the dataset downloads and the expected image and label columns exist.

• Confirm that training completes and the saved model and class-name file are created.

• Upload a valid JPG, JPEG, and PNG image and verify that a prediction is displayed.

• Upload an unsupported file type and confirm that the interface rejects it.

• Check that the app shows a clear message if the trained model is missing.

• Test with images not used during training and record the results.

12. Results and Discussion

Results are intentionally left for completion after running the project. Do not report an assumed accuracy.

Dataset version/source: _____

Number of selected classes: _____

Number of training images: _____

Number of validation/test images: _____

Training epochs: _____

Final training accuracy: _____

Validation/test accuracy: _____

Observed limitations or errors: _____

Insert screenshots of the training output and working Streamlit application in this section.

13. Limitations

The model can only predict among the classes represented during training. Image backgrounds, lighting, leaf angle, image quality, class imbalance, and differences between dataset images and real-world photos can affect performance. A high confidence score does not guarantee that a prediction is correct. The baseline model is intended for learning and demonstration, not for making crop-treatment decisions.

14. Future Enhancements

• Add more plant species and disease classes after validating the dataset pipeline.

• Evaluate with a separate test set and report a confusion matrix and per-class metrics.

• Use data augmentation and transfer learning to improve generalization.

• Add a prediction history page and clearer explanations of supported classes.

• Deploy the application online after checking model size, privacy, and hosting requirements.

15. Conclusion

This project demonstrates an end-to-end deep-learning workflow for plant-leaf image classification: dataset preparation, CNN training, model evaluation, and web-based prediction. Once the code has been run successfully, actual evaluation results and screenshots should be added to complete the report. The application should be treated as an educational prototype rather than a professional diagnostic tool.

References

1. PlantVillage Dataset repository: https://github.com/spMohanty/PlantVillage-Dataset

2. TensorFlow documentation: https://www.tensorflow.org/tutorials/images/classification

3. Streamlit documentation: https://docs.streamlit.io/

4. Hugging Face Datasets documentation: https://huggingface.co/docs/datasets/

5. Scikit-learn documentation: https://scikit-learn.org/stable/